
YILDIZ TECHNICAL UNIVERSITY MECHATRONICS ENGINEERING

Machine Learning Homework 3



Gülsüm Güner 2106A049

Contents

Advanced GUI Application for Interactive Design, Training, and Evaluation of Deep Neural Networks	3
1. Introduction.....	3
2. Network Architecture Configuration and Dynamic Design.....	3
2.1 Fully Connected (Dense) Layers	3
2.2 Convolutional and Pooling Layers (CNNs)	3
2.3 LSTM / GRU Layers for Sequence Data	4
3. Model Saving and Loading	4
4. Training Configuration.....	4
4.1 Optimizers.....	4
4.2 Learning Rate Schedulers.....	5
4.3 Regularization Techniques.....	5
4.4 Early Stopping.....	5
5. Training Visualization and Evaluation.....	5
5.1 Real-Time Training Curves	5
5.2 Final Evaluation Metrics	5
5.3 Weight Gradient Histograms.....	6
6. Transfer Learning and Data Augmentation	6
6.1 Pre-Trained Models	6
6.2 Image Augmentation	7
7. GUI Interface Design	8
8. Conclusion and Contributions	8

Advanced GUI Application for Interactive Design, Training, and Evaluation of Deep Neural Networks

1. Introduction

Deep learning applications have become essential across many domains such as healthcare, finance, autonomous systems, and natural language processing. However, building deep learning models typically requires advanced knowledge in programming and mathematics. The GUI application developed in this work eliminates such barriers by providing users with an intuitive and interactive interface to design, train, and evaluate neural network architectures without writing code.

This application is intended for both academic researchers and industry professionals. It offers a modular, user-friendly interface while integrating advanced deep learning functionalities such as transfer learning, regularization, and real-time visualizations—enabling users to build production-ready models efficiently.

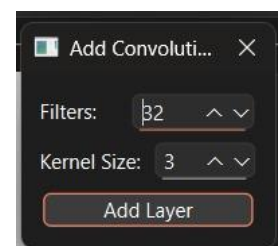
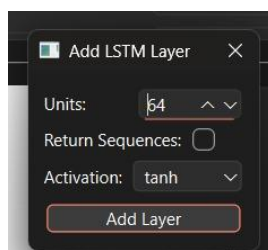
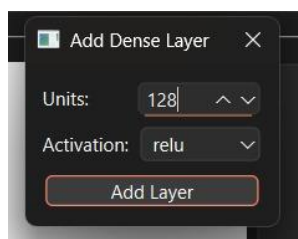
2. Network Architecture Configuration and Dynamic Design

2.1 Fully Connected (Dense) Layers

The GUI allows users to design multilayer perceptron (MLP) networks by specifying:

- The number and size of hidden layers (e.g., $128 \rightarrow 64 \rightarrow 32$),
- Activation functions for each layer:
 - **ReLU**: Fast and widely used, prevents saturation in deep layers.
 - **Sigmoid**: Outputs between 0 and 1, prone to vanishing gradients.
 - **Tanh**: Outputs between -1 and 1, centered activations.

Users can dynamically add or remove layers through the GUI, making it easy to prototype and experiment with different model configurations.

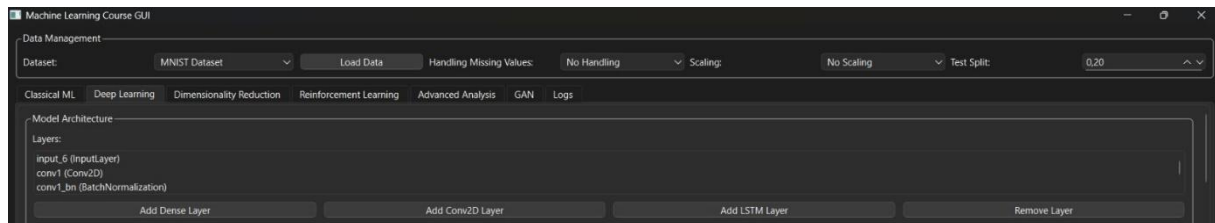


2.2 Convolutional and Pooling Layers (CNNs)

For image-based tasks, convolutional structures can be added via the interface:

- **Conv2D Layers:**

- Configurable number of filters, kernel size (e.g., 3×3, 5×5), and activation functions.
- **Pooling Layers:**
 - Supports MaxPooling and AveragePooling,
 - Adjustable pool sizes,
 - Helps reduce spatial dimensions and increase translation invariance.



2.3 LSTM / GRU Layers for Sequence Data

For time-series and sequential data, users can insert:

- **LSTM (Long Short-Term Memory)** and
- **GRU (Gated Recurrent Unit)** layers.

These layers are ideal for learning long-term dependencies. Configuration options include:

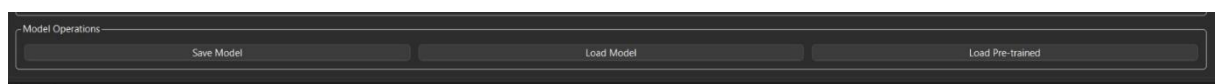
- Number of units,
- Activation function,
- Dropout rate.

3. Model Saving and Loading

Users can save their models as:

- **.h5** files (including both architecture and weights),
- **.json** files (architecture only).

Saved models can be reloaded into the GUI for further training, evaluation, or fine-tuning, promoting reproducibility and model portability.



4. Training Configuration

4.1 Optimizers

The GUI provides multiple optimizers to choose from:

- **SGD (Stochastic Gradient Descent):** Simple and effective.
- **Adam:** Combines momentum and adaptive learning rates; widely used.

- **RMSprop**: Suitable for recurrent networks and noisy problems.

The image shows two screenshots of a 'Training Configuration' GUI. The top screenshot shows the 'Optimizer' dropdown set to 'Adam' with a learning rate of 0.00, dropout rate of 0.20, and L2 Lambda of 0.01. The bottom screenshot shows the 'Optimizer' dropdown menu open, listing 'Adam', 'SGD', and 'RMSprop' (which is highlighted in red). The other settings remain the same.

4.2 Learning Rate Schedulers

Two learning rate scheduling strategies are available:

- **Step Decay**: Reduces learning rate at fixed epoch intervals.
- **Exponential Decay**: Gradually decreases the learning rate at every epoch using an exponential function.

4.3 Regularization Techniques

To prevent overfitting, the GUI includes:

- **Dropout**: Randomly deactivates a portion of neurons during training (user-defined rate).
- **L2 Regularization**: Penalizes large weights to encourage simpler models and better generalization.

4.4 Early Stopping

Training can be automatically stopped if the validation loss does not improve for a given number of epochs. This avoids overfitting and reduces unnecessary computation.

5. Training Visualization and Evaluation

5.1 Real-Time Training Curves

During training, the GUI visualizes:

- **Loss** and
- **Accuracy**
over epochs for both training and validation datasets.

These real-time plots help users understand how the model is learning and detect underfitting or overfitting early.

5.2 Final Evaluation Metrics

Once training is completed, the model is evaluated on the test set, and the GUI displays:

- Accuracy,
- Precision,
- Recall,

- **F1-Score** (particularly useful for imbalanced datasets).

5.3 Weight Gradient Histograms

The GUI displays histograms of weight gradients after each epoch. These visualizations are useful for:

- Diagnosing exploding or vanishing gradients,
- Ensuring healthy model convergence.

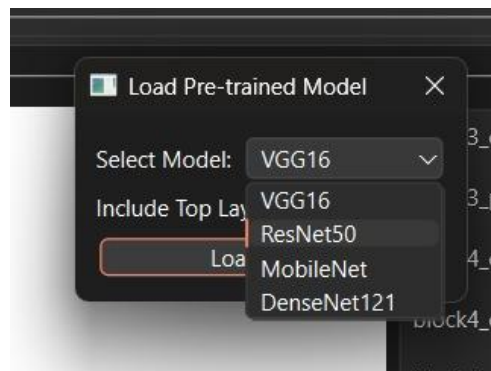
6. Transfer Learning and Data Augmentation

6.1 Pre-Trained Models

Users can import and fine-tune well-known pre-trained models such as:

- **VGG16, ResNet, MobileNet**, etc.

The final layers can be customized via the GUI for specific tasks, allowing users to leverage transfer learning with their own datasets.



Model: "vgg16"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0

Model: "resnet50"

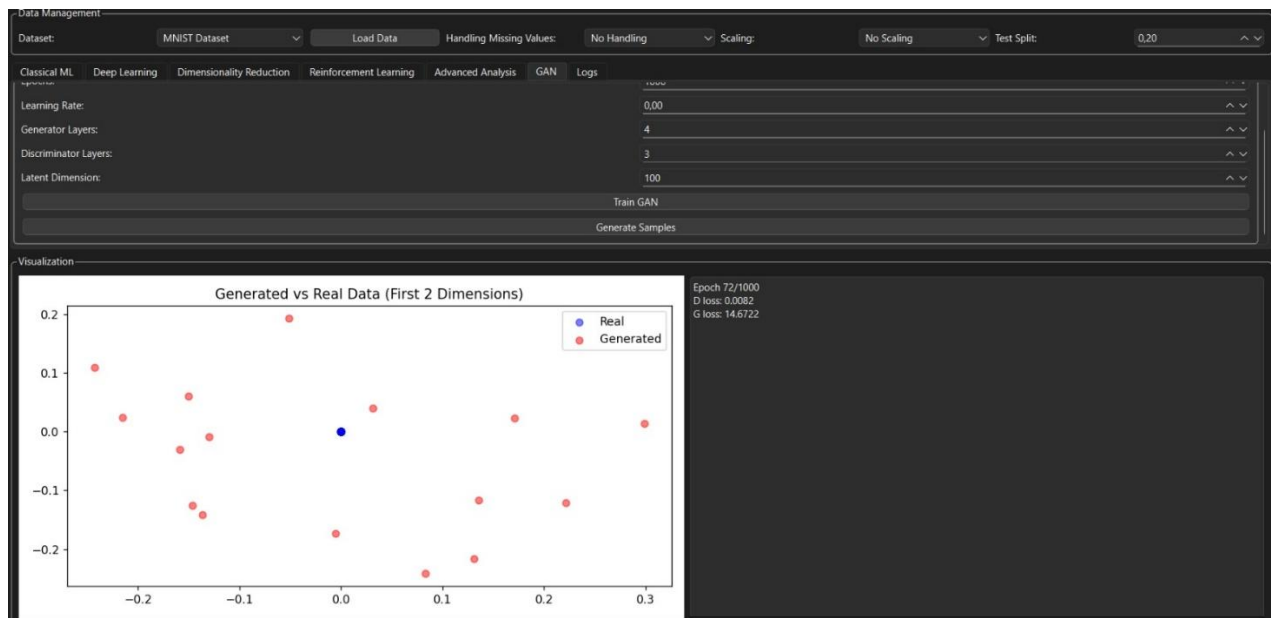
Layer (type)	Output Shape	Param #	Connected to
=====			
input_2 (InputLayer)	[(None, 224, 224, 3)]	0	[]
=====			
conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	['input_2[0][0]']
conv1_conv (Conv2D)	(None, 112, 112, 64)	9472	['conv1_pad[0][0]']
conv1_bn (BatchNormalization)	(None, 112, 112, 64)	256	['conv1_conv[0][0]']
conv1_relu (Activation)	(None, 112, 112, 64)	0	['conv1_bn[0][0]']
pool1_pad (ZeroPadding2D)	(None, 114, 114, 64)	0	['conv1_relu[0][0]']
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	['pool1_pad[0][0]']
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4160	['pool1_pool[0][0]']

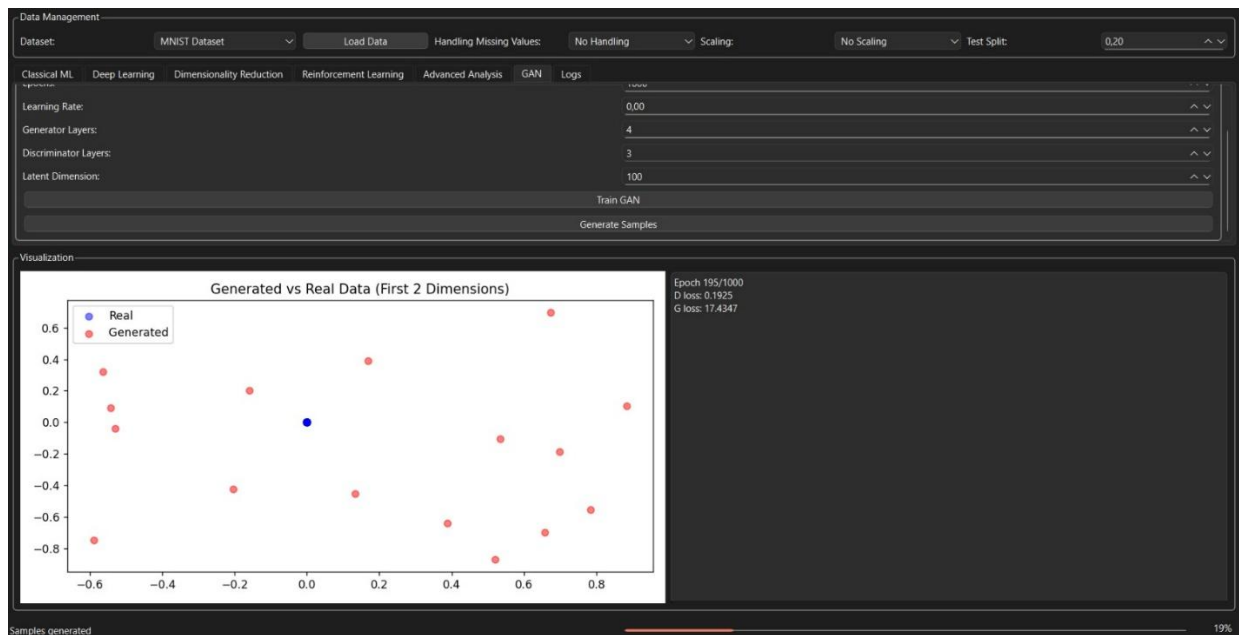
6.2 Image Augmentation

To improve generalization and simulate a larger dataset, the GUI supports:

- **Rotation**
- **Flipping** (horizontal/vertical)
- **Scaling / Zooming**

Augmentation settings can be adjusted interactively before training begins.





7. GUI Interface Design

The GUI is modular and organized into multiple tabs for better usability:

Tab Name	Functionality
Model Design	Add/remove layers, configure architecture and activations
Training Setup	Optimizer selection, learning rate settings, regularization
Visualization	Training curves, metrics, gradient histograms
Data Input	Dataset loading, image augmentation settings
Model Management	Save/load architecture, import pre-trained models

The layout is designed to support users at all expertise levels—from beginners to experienced machine learning engineers.

8. Conclusion and Contributions

This application provides a robust environment for building and evaluating deep learning models interactively. Key contributions include:

- No-code model design with full configurability,
- Integration of advanced training strategies and regularization,
- Real-time feedback and visual diagnostics,

- Support for transfer learning and augmentation,
- Reusability and saving of custom model architectures.

This GUI accelerates deep learning experimentation and promotes better practices in model development and deployment.